

## FIT3182 Assignment 2 Starting Guide

This guide is intended to help you begin Assignment 2. It does not provide a complete solution. Instead, it explains the expected workflow, gives implementation hints, and shows small examples of how to approach stream joins in Spark Structured Streaming. Before reading this guide, please ensure you have carefully read the Assignment specifications, the marking guide, and the enclosed data files.

*Note: Parts of the starting guide were produced using Generative AI based on a reference code.*

### 1. Understand The System You Are Building

The assignment asks you to build an AWAS-style traffic monitoring pipeline.

At a high level, your system should do the following:

1. Read static reference data:

- `vehicle.csv`
- `camera.csv`

2. Stream live camera events from:

- `camera_event_A.csv`
- `camera_event_B.csv`
- `camera_event_C.csv`

3. Publish those events to Kafka using producers.

4. Consume the Kafka streams using Spark Structured Streaming.

5. Detect:

- instantaneous speed violations at a camera,
- average speed violations between camera checkpoints.

6. Store detected violations in MongoDB.

7. Produce meaningful visualisations from the stored results.

8. Explain your design and implementation clearly in markdown.

The key idea is that the CSV files simulate camera events. Kafka represents live ingestion. Spark performs stateful stream processing. MongoDB stores the detected violation records.

### 2. Recommended Submission Structure

Use the structure recommended in the assignment specification. You may combine some notebooks if your workflow is clear, but your README should explain exactly how to run your code.

### 3. Step 1: Inspect The Data

Before writing streaming code, inspect the CSV files.

You should identify:

- the columns in each file,
- the data types you expect,
- whether each camera event file corresponds to one camera,
- the meaning of `batch_id`,
- timestamp format,
- speed-limit information in `camera.csv`.

Example fields in the event streams include:

```
event_id, batch_id, car_plate, camera_id, timestamp, speed_reading
```

Important checks:

- Are timestamps ordered?
- Do matching vehicles appear across multiple camera files?
- Are camera IDs consistent with `camera.csv`?
- Are speed readings numeric?
- Are there duplicate vehicle plates in `vehicle.csv`?

These checks will help you avoid many later errors.

## 4. Step 2: Design MongoDB Collections

At a minimum, design these collections:

- `vehicles`
- `cameras`
- `violations`

For each collection, document:

- purpose,
- schema,
- sample document,
- indexes,
- whether you would shard it,
- retention policy, if relevant.

### Example: Vehicle Collection

```
{
  "car_plate": "ABC123",
  "owner_name": "Example Name",
  "owner_addr": "Example Address",
  "vehicle_type": "Sedan",
  "registration_date": ISODate("2024-01-01T00:00:00Z")
}
```

Useful index:

```
{ "car_plate": 1 }
```

### Example: Camera Collection

```
{
  "camera_id": 1,
  "location": {
    "type": "Point",
    "coordinates": [102.6601002, 2.157730731]
  },
  "position": 152.5,
  "speed_limit": 110
}
```

Useful indexes:

```
{ "camera_id": 1 }
{ "location": "2dsphere" }
```

### Example: Violation Collection

One reasonable design is to keep one daily record per vehicle, with an array of violations for that day.

```
{
  "car_plate": "ABC123",
  "date": ISODate("2024-01-01T00:00:00Z"),
  "violations": [
    {
      "violation_type": "INSTANTANEOUS",
      "camera_id_start": 1,
      "camera_id_end": 1,
      "timestamp_start": ISODate("2024-01-01T08:00:04Z"),
      "timestamp_end": ISODate("2024-01-01T08:00:04Z"),
      "speed_reading": 125.0
    },
    {
      "violation_type": "AVERAGE",
      "camera_id_start": 1,
      "camera_id_end": 2,
      "timestamp_start": ISODate("2024-05-01T14:30:00Z"),
      "timestamp_end": ISODate("2024-05-01T14:35:00Z"),
      "speed_reading": 115.0
    }
  ]
}
```

Useful compound index:

```
{ "car_plate": 1, "date": 1 }
```

In your explanation, justify whether you reference or embed data. For example, storing `car_plate` in a violation record references the vehicle collection, while storing a list of violation events inside a daily document is embedding.

## 5. Step 3: Build Kafka Producers

Each producer should read a single event CSV file and publish the records to a Kafka topic.

Typical topics:

```
camera-events-A
camera-events-B
camera-events-C
```

Your producer should:

- group records by `batch_id`,
- send each batch every `n` seconds,
- include metadata such as `producer_id`,
- convert numeric fields to suitable types,
- handle malformed rows gracefully,
- optionally save a checkpoint to resume.

Example event payload:

```
event = {
    "event_id": row["event_id"],
    "batch_id": int(row["batch_id"]),
    "car_plate": row["car_plate"],
    "camera_id": int(row["camera_id"]),
    "timestamp": row["timestamp"],
    "speed_reading": float(row["speed_reading"]),
    "producer_id": "A"
}
```

You may send the timestamp as an ISO string or as a Unix timestamp. Whichever you choose, keep the schema consistent in your Spark consumer.

## 6. Step 4: Read Kafka Streams In Spark

In Spark Structured Streaming, you normally read from Kafka as a stream of key/value messages. The Kafka value is usually a JSON string, so you must parse it.

Example schema:

```
from pyspark.sql.types import StructType, StructField, StringType, IntegerType, DoubleType

event_schema = StructType([
    StructField("event_id", StringType()),
    StructField("batch_id", IntegerType()),
    StructField("car_plate", StringType()),
    StructField("camera_id", IntegerType()),
    StructField("timestamp", StringType()),
    StructField("speed_reading", DoubleType()),
    StructField("producer_id", StringType())
])
```

Example Kafka stream reader:

```
from pyspark.sql.functions import col, from_json, to_timestamp

raw_stream = (
    spark.readStream
    .format("kafka")
    .option("kafka.bootstrap.servers", "localhost:9092")
    .option("subscribe", "camera-events-A")
```

```

        .option("startingOffsets", "latest")
        .load()
    )

    camera_a_stream = (
        raw_stream
        .selectExpr("CAST(value AS STRING) AS json_value")
        .select(from_json(col("json_value"), event_schema).alias("data"))
        .select("data.*")
        .withColumn("event_time", to_timestamp(col("timestamp")))
    )

```

Repeat this pattern for streams B and C.

## 7. Step 5: Detect Instantaneous Violations

An instantaneous violation occurs when:

`speed_reading > speed limit of the recording camera`

You can load speed limits from `camera.csv` or MongoDB. For a small number of cameras, a dictionary is acceptable:

```

speed_limits = {
    1: 110,
    2: 110,
    3: 90
}

```

In streaming code, you can filter events:

```

from pyspark.sql.functions import udf
from pyspark.sql.types import BooleanType

def is_speeding(camera_id, speed):
    return speed > speed_limits.get(camera_id, float("inf"))

is_speeding_udf = udf(is_speeding, BooleanType())

instant_violations = camera_a_stream.filter(
    is_speeding_udf(col("camera_id"), col("speed_reading"))
)

```

You should apply similar logic to all camera streams.

## 8. Step 6: Understand Stream Joins

Average speed detection requires matching the same vehicle across two cameras.

For example:

- camera A event: car `ABC123` seen at camera 1 at `08:00:00`
- camera B event: car `ABC123` seen at camera 2 at `08:00:30`

If the camera distance is 1 km, the average speed is:

```

average speed = distance/time
               = 1 km / (30 seconds / 3600)
               = 120 km/h

```

If the ending camera speed limit is 110 km/h, this is an average-speed violation.

## 9. Example: A-B Stream Join

The following is a simplified example. You will need to adapt it to your schema and environment.

```

from pyspark.sql.functions import expr, col

watermarked_a = camera_a_stream.withWatermark("event_time", "10 minutes")
watermarked_b = camera_b_stream.withWatermark("event_time", "10 minutes")

joined_ab = (
    watermarked_a.alias("a")
    .join(
        watermarked_b.alias("b"),
        expr("""
            a.car_plate = b.car_plate AND
            a.event_time < b.event_time AND
            b.event_time <= a.event_time + interval 60 seconds
        """),
        "inner"
    )
    .select(
        col("a.car_plate").alias("car_plate"),
        col("a.camera_id").alias("camera_id_start"),
        col("b.camera_id").alias("camera_id_end"),
        col("a.event_time").alias("timestamp_start"),
        col("b.event_time").alias("timestamp_end"),
        col("a.speed_reading").alias("speed_start"),
        col("b.speed_reading").alias("speed_end")
    )
)

```

This join matches records where:

- The same car appears in both streams,
- The second event occurs after the first event,
- The second event occurs within a chosen time window.

The time window is important. Without it, Spark may need to keep too much state.

## 10. Example: Calculating Average Speed After A Join

Assume cameras 1 and 2 are 1 km apart.

```

from pyspark.sql.functions import unix_timestamp, lit

distance_km = 1.0
ending_camera_limit = 110

ab_with_avg_speed = (
    joined_ab
    .withColumn(

```

```

        "travel_time_hours",
        (unix_timestamp(col("timestamp_end")) - unix_timestamp(col("timestamp_start"))) /
        lit(3600.0)
    )
    .withColumn(
        "avg_speed",
        lit(distance_km) / col("travel_time_hours")
    )
)

ab_avg_violations = ab_with_avg_speed.filter(
    col("avg_speed") > lit(ending_camera_limit)
)

```

In your final assignment, avoid hardcoding where possible. It is better to calculate distances from `camera.csv`, or at least clearly document any assumptions.

## 11. Example: B-C Stream Join

The B-C join follows the same pattern, and the A-B join is best attempted on your own.

Then calculate the average speed using the distance between cameras 2 and 3, and compare against the ending camera's speed limit.

## 12. Inner Join, Outer Join, And Dropped Records

An inner join only returns matched pairs. This is usually easiest when first developing.

However, the marking guide rewards more advanced handling of unmatched or expired records. To reason about that, consider:

- What happens if a car appears at camera A but not camera B?
- What happens if the B event arrives very late?
- What happens if timestamps are out of order?
- What happens if the same car appears multiple times within the join window?

Possible strategies:

- Start with an inner join to prove your average-speed logic works.
- Add watermarks to limit the state.
- Log unmatched, invalid, or expired records.
- Document your assumptions about timing tolerance.

For example, you might discard a pair if:

```
timestamp_end <= timestamp_start
```

or if:

```
timestamp_end is more than 60 seconds after timestamp_start
```

The exact value should be justified using the camera distance, expected speed range, and dataset behaviour.

### 13. Step 7: Write Violations To MongoDB

Spark can write streaming output using `foreachBatch`. This is often practical for MongoDB writes.

Simplified pattern:

```
def write_violations_to_mongo(batch_df, batch_id):
    rows = batch_df.collect()

    for row in rows:
        violation_doc = {
            "car_plate": row.car_plate,
            "date": row.timestamp_start.date().isoformat(),
            "violation": {
                "camera_id_start": row.camera_id_start,
                "camera_id_end": row.camera_id_end,
                "timestamp_start": row.timestamp_start,
                "timestamp_end": row.timestamp_end,
                "speed_reading": row.avg_speed,
                "violation_type": "AVERAGE"
            }
        }

        # Insert or update MongoDB here.
```

A stronger implementation should use:

- `update_one(..., upsert=True)` to merge daily records,
- `$push` or `$addToSet` to append violation details,
- retry handling for failed writes,
- bulk writes for efficiency.

Example upsert idea:

```
db.violations.update_one(
    {
        "car_plate": car_plate,
        "date": violation_date
    },
    {
        "$setOnInsert": {
            "car_plate": car_plate,
            "date": violation_date
        },
        "$push": {
            "violations": violation_detail
        }
    },
    upsert=True
)
```

Be careful with duplicate processing. Streaming systems may retry batches, so think about idempotency.

### 14. Step 8: Visualisation

Your visualisations should answer operational questions, not just display data.



Possible visualisations:

- violation count over time,
- average violation speed over time,
- separate lines for instantaneous and average-speed violations,
- camera-level violation counts,
- segment-level average-speed violation counts,
- annotated peaks, sudden drops, or unusual periods.

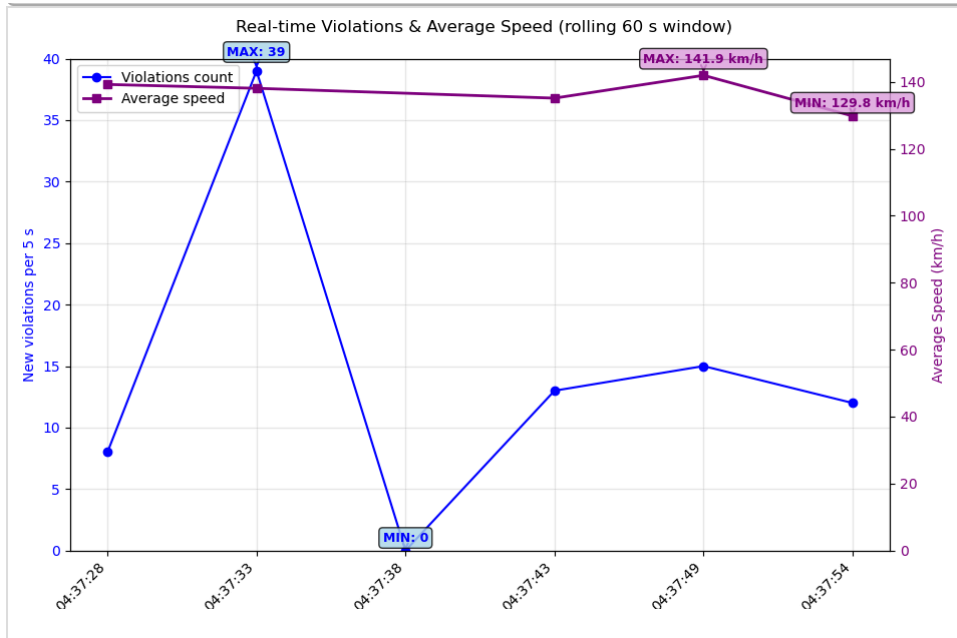
Minimum expectations:

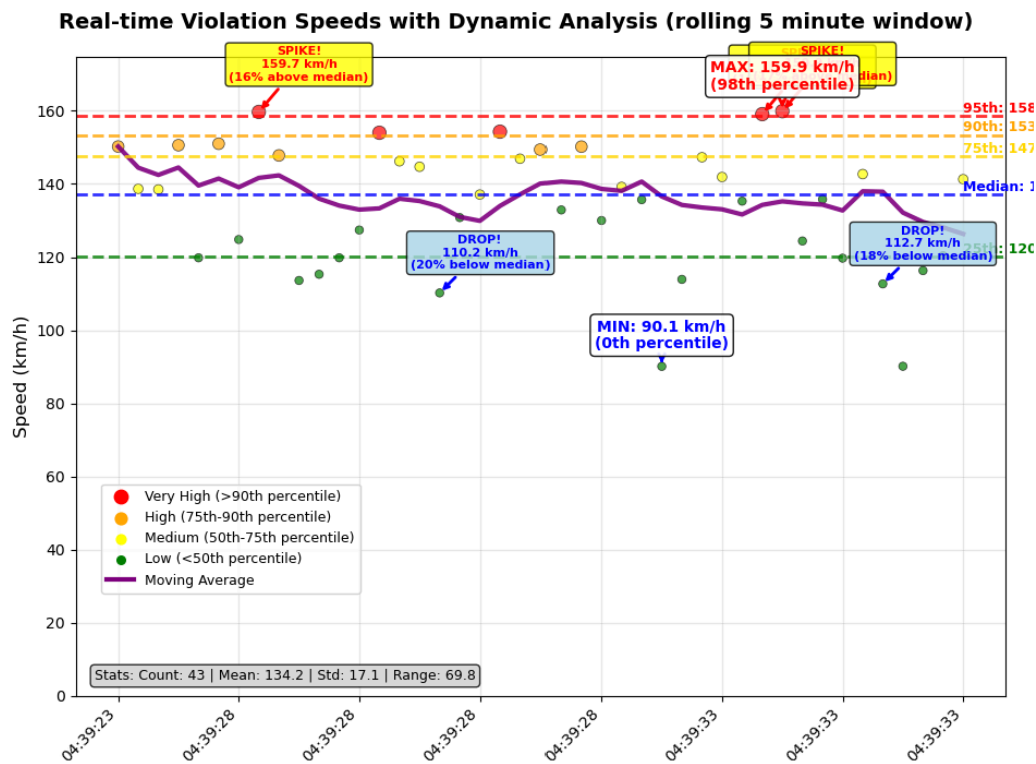
- clear title,
- labelled axes,
- legend,
- readable line styles or markers,
- annotations for interesting points.

Example analytical questions:

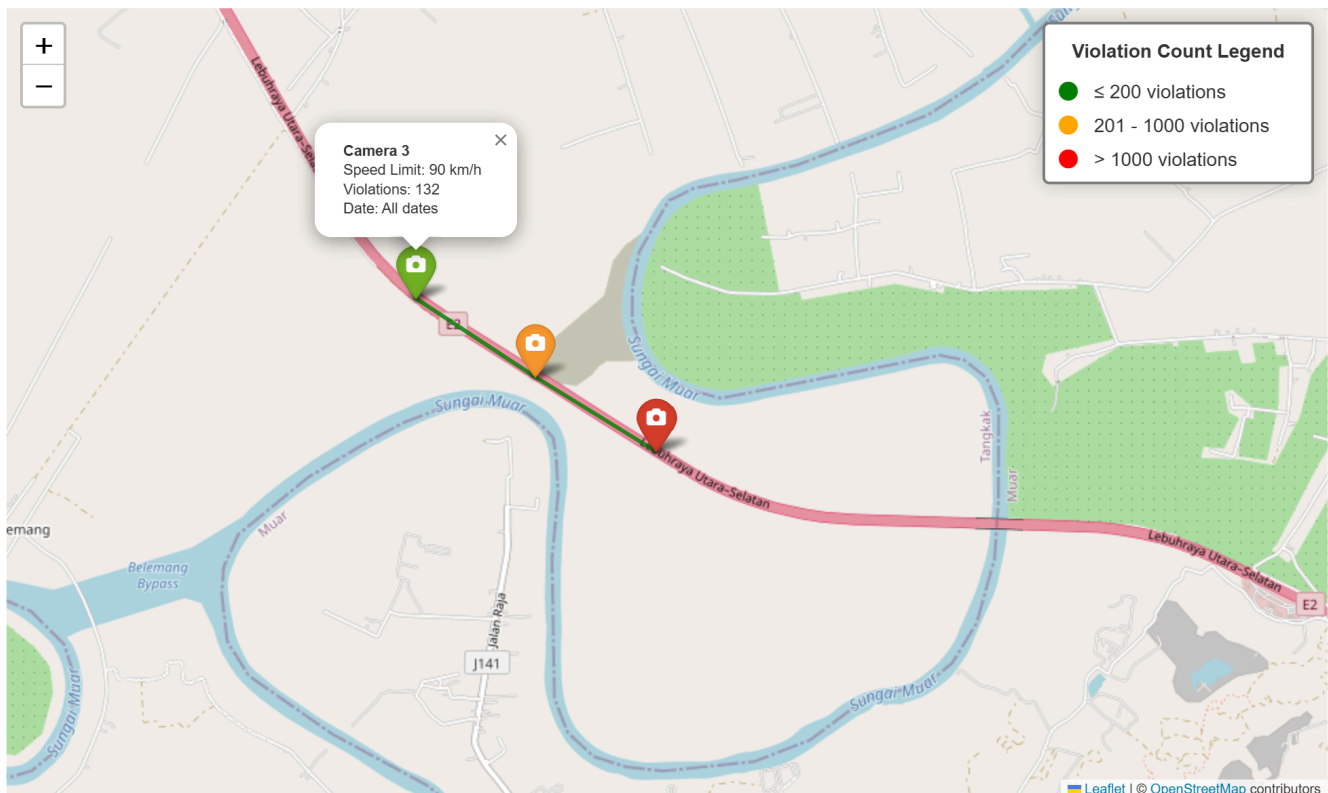
- When do violations peak?
- Which camera or segment has the most violations?
- Are average-speed violations more common than instantaneous violations?
- Are there sudden spikes that might indicate unusual traffic behaviour?

Sample plots:





Date: All | Total violations: 1688



Important - Please try not to follow the sample plots exactly provided above. Students aiming for HD should attempt to include interactive plots to provide a better scale of visualisation.

## 15. Step 9: Documentation

Your notebooks should explain:

- How do your producers work?
- How are your Kafka topics organised?
- How are your Spark streams parsed?
- How does your stream join work?
- How do you calculate average speed?
- How do you handle late, unmatched, or invalid records?
- How do you write to MongoDB?
- How do your visualisations support operational decisions?

Useful additions:

- Architecture diagram,
- Table of important parameters,
- Sample input/output records,
- Assumptions and limitations,
- Brief explanation of indexes and schema design.

Example parameter table:

| Parameter           | Example            | Purpose                                    |
|---------------------|--------------------|--------------------------------------------|
| Producer sleep time | 5 seconds          | Controls event publishing rate             |
| Watermark           | 10 minutes         | Limits retained streaming state            |
| Join window         | 60 seconds         | Defines valid camera-to-camera travel time |
| MongoDB daily key   | `car_plate + date` | Merges same-day violations                 |